

Лабораторная работа №7

**Дискретное логарифмирование в конечном поле. р-метод
Полларда**

Кюнкриков Даниил Саналович

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	ρ -метод Полларда для DLP	7
4	Выполнение лабораторной работы	8
	Список литературы	12

Список иллюстраций

Список таблиц

1 Цель работы

Цель работы - Реализовать ρ -метод Полларда для решения задачи дискретного логарифмирования $a^x \equiv b \pmod{p}$ и проверить корректность реализации на тестовых данных.

2 Задание

1. Рассмотреть постановку задачи DLP в $\mathbb{F}_p$.
2. Изучить идею р-метода Полларда (случайное блуждание, поиск коллизии методом Флойда).
3. Реализовать алгоритм на Python и протестировать на контрольном примере.

3 Теоретическое введение

Задача дискретного логарифмирования (DLP) в группе вычетов по модулю простого p формулируется так: для заданных a и b найти x , удовлетворяющее

$$a^x \equiv b \pmod{p}.$$

Сложность решения DLP лежит в основе ряда криптографических протоколов (например, схем Диффи—Хеллмана и Эль-Гамала) [1; 2].

3.1 ρ -метод Полларда для DLP

Метод Полларда для вычисления индекса строит псевдослучайное блуждание по группе и использует поиск коллизии (Floyd). Каждый элемент последовательности представляется в виде:

$$c \equiv a^\alpha b^\beta \pmod{p},$$

и при совпадении двух значений c получается линейное сравнение относительно x по модулю порядка подгруппы r [1; 3].

Ожидаемая трудоёмкость метода порядка $O(\sqrt{r})$, что делает его базовой атакой для оценки параметров групп в криптографии [2; 3].

4 Выполнение лабораторной работы

Программная реализация расширенного алгоритма Евклида показана на Листинге №1.

4.0.1 Листинг 1 — Расширенный алгоритм Евклида

```
def egcd(a: int, b: int):  
    if b == 0:  
        return (a, 1, 0)  
    g, y1, x1 = egcd(b, a % b)  
    return (g, y1, x1 - (a // b) * y1)
```

Программная реализация р-метода Полларда для DLP показана на Листинге №2.

4.0.2 Листинг 2 — р-метод Полларда для дискретного логарифма

```
import secrets  
  
def rho_dlog(p: int, a: int, b: int, r: int, max_steps: int = 1_000_000):  
    """Ищет x из  $a^x \equiv b \pmod{p}$ , где r — порядок a (или порядок подгруппы)."""
```

```

half = p // 2

def step(x: int, alpha: int, beta: int):
    """Один шаг f(x) + согласованное обновление (alpha, beta)."""
    if x < half:
        # x <- a*x
        return (a * x) % p, (alpha + 1) % r, beta
    else:
        # x <- b*x
        return (b * x) % p, alpha, (beta + 1) % r

while True:
    # случайная инициализация
    u = secrets.randbelow(r)
    v = secrets.randbelow(r)

    c = (pow(a, u, p) * pow(b, v, p)) % p
    d = c

    alpha1, beta1 = u, v
    alpha2, beta2 = u, v

    for _ in range(max_steps):
        # черепаха: 1 шаг
        c, alpha1, beta1 = step(c, alpha1, beta1)

        # заяц: 2 шага
        d, alpha2, beta2 = step(d, alpha2, beta2)

```

```

d, alpha2, beta2 = step(d, alpha2, beta2)

if c == d:
    # (beta1 - beta2) * x ≡ (alpha2 - alpha1) (mod r)
    A = (beta1 - beta2) % r
    delta = (alpha2 - alpha1) % r

    g, xcoef, _ = egcd(A, r)
    if delta % g != 0:
        # неудачный запуск – перезапустим с другими (u, v)
        break

    r_red = r // g
    x = (xcoef * (delta // g)) % r_red
    return x

# если по max_steps не нашли корректное решение – повторяем с новыми u, v

```

Код запуска программы показан на Листинге №3.

4.0.3 Листинг 3 — Ввод параметров и запуск

```

if __name__ == "__main__":
    print("Введите p, a, b, r через пробел (или 'выход')")

    while True:
        s = input("> ").strip().lower()
        if s == "выход":
            break

```

```

try:
    p, a, b, r = map(int, s.split())
    x = rho_dlog(p, a, b, r)
    print("Invalid problem" if x is None else f"x = {x}")
except Exception:
    print("Ошибка ввода. Формат: p a b r")

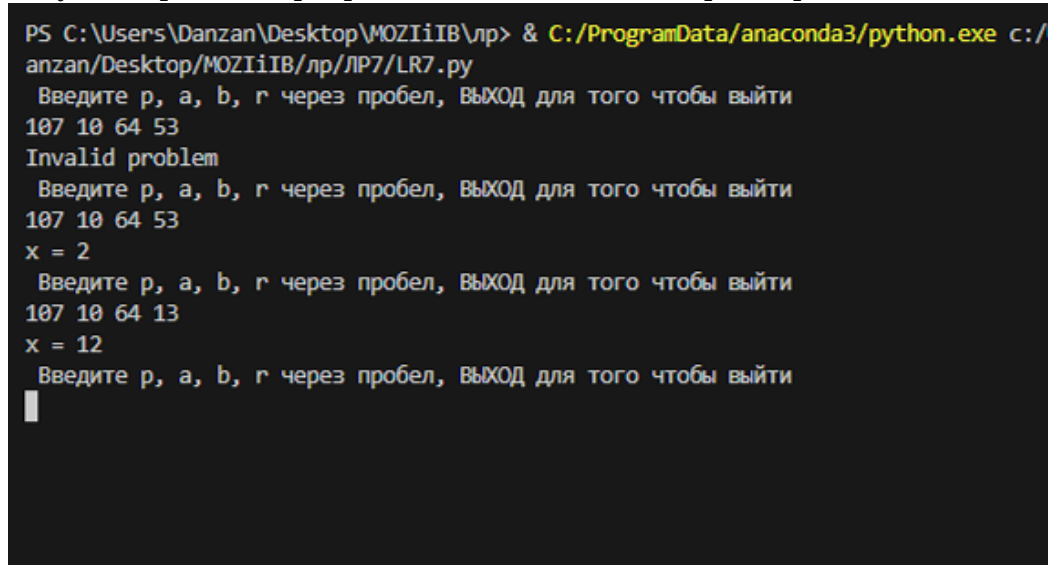
```

Проверим контрольный пример: $p = 107$, $a = 10$, $b = 64$, $r = 53$.

Ожидаемый результат: $x = 20$, так как $10^{20} \equiv 64 \pmod{107}$.

Пример вывода программы:

Результат работы программы для заданных параметров показан на ?@fig-1.



ВОДЫ

В ходе работы:

- рассмотрена постановка задачи дискретного логарифмирования в конечном поле;
- реализован ρ -метод Полларда для DLP с поиском коллизии методом Флойда;
- проверена корректность реализации на контрольных параметрах.

Список литературы

1. *Galbraith S. D.* Mathematics of Public Key Cryptography. — Cambridge University Press, 2012. — ISBN 1-107-01392-5.
2. *Menezes A. J., van Oorschot P. C., Vanstone S. A.* Handbook of Applied Cryptography. — CRC Press, 1996. — ISBN 0-8493-8523-7.
3. *Pollard J. M.* Monte Carlo Methods for Index Computation (Mod p) // Mathematics of Computation. — 1978. — Т. 32, № 143. — С. 918—924.